

Rapport Technique

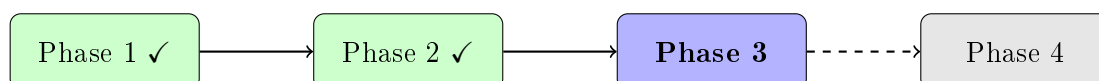
Phase 3 : Architecture et Entraînement du Modèle GNN

Projet NYC Traffic Prediction

Prédiction de Trafic Urbain par Graph Neural Network
avec Capacité de Transfer Learning

Script : `train_gnn.py`

Date 6 janvier 2026
Version 1.0.0
Auteur Caramella Enzo



Contents

1	Introduction	3
1.1	Objectif	3
1.2	Entrées depuis la Phase 2	3
1.3	Contraintes Techniques	3
2	Architectures GNN Implémentées	3
2.1	GraphSAGE (Recommandé)	3
2.1.1	Principe	3
2.1.2	Avantages pour le Transfer Learning	4
2.1.3	Architecture Implémentée	4
2.2	Graph Attention Network (GAT)	4
2.2.1	Mécanisme d'Attention	4
2.2.2	Configuration	5
2.3	Graph Convolutional Network (GCN)	5
2.4	Comparaison des Architectures	5
3	Stratégie de Split des Données	5
3.1	Split Spatial (Recommandé)	5
3.1.1	Justification	6
3.1.2	Implémentation	6
3.2	Split Aléatoire (Alternative)	6
4	Gestion des Données Partielles	6
4.1	Masquage des Labels	6
4.2	Pondération par Observations	6
4.3	Normalisation des Labels	7
5	Entraînement	7
5.1	Configuration par Défaut	7
5.2	Optimiseur et Scheduler	7
5.3	Early Stopping	7
5.4	Métriques d'Évaluation	8
6	Fichiers de Sortie	8
6.1	Modèle	8
6.2	Configuration	8
6.3	Historique d'Entraînement	9
6.4	Résultats Finaux	9
7	Utilisation	9
7.1	Installation des Dépendances	9
7.2	Exemples d'Exécution	9
7.3	Arguments Disponibles	10
8	Code Source : Extraits Clés	10
8.1	Classe GraphSAGE	10
8.2	Loss Pondérée	11
9	Prochaines Étapes	11
9.1	Phase 4 : Validation et Transfer Learning	11
9.2	Améliorations Possibles	12

10 Conclusion	12
A Formules Mathématiques Détaillées	12
A.1 GraphSAGE - Agrégation Mean	12
A.2 GAT - Coefficients d'Attention	12
A.3 GCN - Propagation	13
B Nombre de Paramètres	13

1 Introduction

Ce rapport documente l'implémentation de la **Phase 3** du projet de prédiction de trafic urbain : la conception et l'entraînement du modèle Graph Neural Network (GNN).

1.1 Objectif

L'objectif de cette phase est de développer un modèle GNN capable de :

- Prédire le **temps de trajet** sur chaque segment routier
- Apprendre des représentations **inductives** (transférables)
- Exploiter la **structure topologique** du réseau routier
- Gérer les **données partielles** (22.3% de couverture)

1.2 Entrées depuis la Phase 2

Le script utilise les fichiers générés par la Phase 2 (Line Graph) :

Table 1: Fichiers d'entrée depuis Phase 2

Fichier	Shape	Description
edge_index.npy	(2, 805415)	Matrice d'adjacence COO
node_features.npy	(260855, 14)	Features des nœuds
mean_travel_times.npy	(260855,)	Labels (temps moyens)
observation_counts.npy	(260855,)	Poids d'observation
node_mapping.csv	260855 lignes	Coordonnées spatiales

1.3 Contraintes Techniques

- **Mémoire limitée** : 8 GB RAM
- **Couverture partielle** : Seulement 22.3% des liens ont des données de trafic
- **Transférabilité** : Le modèle doit être applicable à d'autres villes

2 Architectures GNN Implémentées

Trois architectures de Graph Neural Networks ont été implémentées pour permettre une comparaison et offrir différents compromis entre performance et complexité.

2.1 GraphSAGE (Recommandé)

GraphSAGE (SAmple and aggreGatE) est l'architecture recommandée pour le transfer learning car elle apprend à **agrégier les features des voisins** de manière inductive.

2.1.1 Principe

Pour chaque nœud v , GraphSAGE apprend une fonction d'agrégation :

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, \text{AGG} \left(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\} \right) \right) \right) \quad (1)$$

où :

- $h_v^{(k)}$: représentation du nœud v à la couche k
- $\mathcal{N}(v)$: voisinage du nœud v
- AGG : fonction d'agrégation (mean, max, LSTM)
- σ : fonction d'activation (ReLU)

2.1.2 Avantages pour le Transfer Learning

1. **Inductif** : Ne dépend pas de la structure exacte du graphe
2. **Échantillonnage** : Utilise un sous-ensemble de voisins (scalable)
3. **Généralisation** : Peut traiter des nœuds jamais vus

2.1.3 Architecture Implémentée

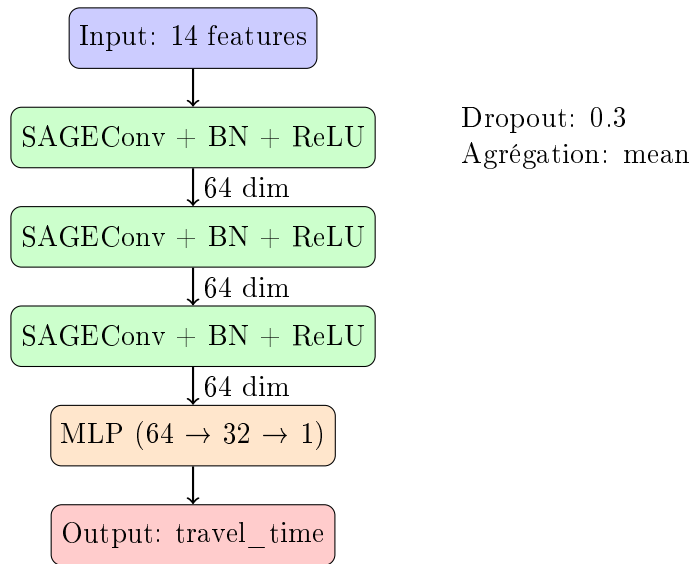


Figure 1: Architecture GraphSAGE implémentée

2.2 Graph Attention Network (GAT)

GAT utilise un mécanisme d'**attention** pour pondérer l'importance de chaque voisin.

2.2.1 Mécanisme d'Attention

Les coefficients d'attention sont calculés par :

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\vec{a}^T [Wh_i || Wh_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\vec{a}^T [Wh_i || Wh_k]))} \quad (2)$$

La mise à jour du nœud utilise l'attention multi-têtes :

$$h'_i = \bigoplus_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k W^k h_j \right) \quad (3)$$

2.2.2 Configuration

- **Têtes d'attention** : 4 (par défaut)
- **Dimension par tête** : $64/4 = 16$
- **Activation** : ELU

2.3 Graph Convolutional Network (GCN)

GCN est inclus comme **baseline** mais est moins adapté au transfer learning car il est transductif.

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right) \quad (4)$$

où $\tilde{A} = A + I$ et $\tilde{D}$ est la matrice de degré correspondante.

2.4 Comparaison des Architectures

Table 2: Comparaison des architectures GNN

Critère	GraphSAGE	GAT	GCN
Inductif	✓	✓	×
Attention	×	✓	×
Scalabilité	Excellente	Bonne	Moyenne
Transfer Learning	Recommandé	Bon	Limité
Complexité mémoire	$O(\mathcal{N} \cdot d)$	$O(K \cdot \mathcal{E} \cdot d)$	$O(\mathcal{E} \cdot d)$

3 Stratégie de Split des Données

La validation du modèle utilise un **split spatial** pour simuler le scénario de transfer learning.

3.1 Split Spatial (Recommandé)



Figure 2: Split spatial basé sur la longitude

3.1.1 Justification

Le split spatial simule le scénario réel de transfer learning :

- Le modèle est entraîné sur une partie de la ville (Manhattan, Brooklyn)
- Il est évalué sur une zone **géographiquement distincte** (Queens)
- Si la validation sur Queens est concluante, le transfer vers Marseille est viable

3.1.2 Implémentation

```

1 def create_spatial_split(data, train_ratio=0.7, val_ratio=0.15):
2     coords = data.coords.numpy()
3     x_coords = coords[:, 0] # Longitude
4
5     # Percentiles pour le decoupage
6     train_threshold = np.percentile(x_coords, train_ratio * 100)
7     val_threshold = np.percentile(x_coords, (train_ratio + val_ratio) * 100)
8
9     # Masques
10    train_mask = x_coords <= train_threshold
11    val_mask = (x_coords > train_threshold) & (x_coords <= val_threshold)
12    test_mask = x_coords > val_threshold
13
14    return train_mask, val_mask, test_mask

```

Listing 1: Split spatial basé sur les coordonnées

3.2 Split Aléatoire (Alternative)

Un split aléatoire est disponible pour comparaison, mais ne valide pas la capacité de généralisation spatiale.

4 Gestion des Données Partielles

Un défi majeur est la **couverture partielle** des données : seulement 22.3% des segments routiers ont des données de trafic.

4.1 Masquage des Labels

$$\mathcal{L}_{\text{masked}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \ell(y_i, \hat{y}_i) \quad (5)$$

où $\mathcal{M} = \{i : y_i \neq \text{NaN}\}$ est l'ensemble des nœuds avec labels valides.

4.2 Pondération par Observations

Les segments avec plus d'observations sont plus fiables. La loss pondérée est :

$$\mathcal{L}_{\text{weighted}} = \sum_{i \in \mathcal{M}} w_i \cdot \ell(y_i, \hat{y}_i) \quad (6)$$

avec les poids normalisés en log pour atténuer les outliers :

$$w_i = \frac{\log(1 + n_i)}{\sum_{j \in \mathcal{M}} \log(1 + n_j)} \quad (7)$$

où n_i est le nombre d'observations pour le segment i .

4.3 Normalisation des Labels

Les temps de trajet sont normalisés par Z-score sur le train set :

$$\tilde{y}_i = \frac{y_i - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (8)$$

Les paramètres μ_{train} et σ_{train} sont sauvegardés pour l'inférence.

5 Entraînement

5.1 Configuration par Défaut

Table 3: Hyperparamètres par défaut

Paramètre	Valeur	Description
<i>Architecture</i>		
model_type	sage	GraphSAGE recommandé
hidden_channels	64	Dimension cachée
num_layers	3	Profondeur du réseau
dropout	0.3	Régularisation
heads	4	Têtes d'attention (GAT)
<i>Entraînement</i>		
epochs	200	Maximum d'epochs
learning_rate	0.001	Taux d'apprentissage
weight_decay	10^{-4}	Régularisation L2
patience	20	Early stopping
<i>Loss</i>		
loss_type	mse	Mean Squared Error
use_weights	True	Pondération par observations

5.2 Optimiseur et Scheduler

- **Optimiseur** : AdamW avec weight decay
- **Scheduler** : ReduceLROnPlateau (facteur 0.5, patience 10)
- **Gradient clipping** : max_norm = 1.0

5.3 Early Stopping

L'entraînement s'arrête si la validation loss ne s'améliore pas pendant **patience** epochs :

Algorithm 1 Early Stopping

```

1: best_loss ← ∞
2: counter ← 0
3: for epoch = 1 to max_epochs do
4:   Train one epoch
5:   val_loss ← Evaluate on validation set
6:   if val_loss < best_loss − δ then
7:     best_loss ← val_loss
8:     counter ← 0
9:     Save model checkpoint
10:  else
11:    counter ← counter + 1
12:  end if
13:  if counter ≥ patience then
14:    break
15:  end if
16: end for
17: Restore best model

```

5.4 Métriques d'Évaluation

Table 4: Métriques calculées

Métrique	Formule	Interprétation
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Erreur quadratique moyenne
RMSE	$\sqrt{\text{MSE}}$	En secondes (même unité que y)
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	Erreur absolue moyenne
MAPE	$\frac{100}{n} \sum \left \frac{y_i - \hat{y}_i}{y_i} \right $	Erreur relative en %

6 Fichiers de Sortie

Le script génère les fichiers suivants dans le répertoire de sortie :

6.1 Modèle

- `model_best.pt` : Checkpoint du meilleur modèle
 - `model_state_dict` : Poids du modèle
 - `optimizer_state_dict` : État de l'optimiseur
 - `y_mean`, `y_std` : Paramètres de normalisation
 - `config` : Configuration complète

6.2 Configuration

```

1 {
2   "model_type": "sage",
3   "hidden_channels": 64,
4   "num_layers": 3,
5   "dropout": 0.3,
6   "in_channels": 14,

```

```

7  "y_mean": 45.23,
8  "y_std": 28.15
9  }

```

Listing 2: Structure de config.json

6.3 Historique d'Entraînement

Le fichier `training_history.json` contient :

- `train_loss` : Liste des losses d'entraînement
- `val_loss` : Liste des losses de validation
- `train_metrics` : Métriques par epoch (train)
- `val_metrics` : Métriques par epoch (validation)
- `lr` : Évolution du learning rate

6.4 Résultats Finaux

```

1  {
2      "config": {...},
3      "train_samples": 40787,
4      "val_samples": 8740,
5      "test_samples": 8740,
6      "best_val_loss": 0.0234,
7      "test_metrics": {
8          "rmse": 12.45,
9          "mae": 8.32,
10         "mape": 18.5
11     },
12     "training_time_minutes": 45.2
13 }

```

Listing 3: Structure de final_results.json

7 Utilisation

7.1 Installation des Dépendances

```

1  # PyTorch (selon votre configuration CUDA)
2  pip install torch torchvision torchaudio
3
4  # PyTorch Geometric
5  pip install torch-geometric
6
7  # Dépendances additionnelles
8  pip install numpy pandas scipy

```

7.2 Exemples d'Exécution

```

1  # Entraînement basique avec GraphSAGE
2  python train_gnn.py \
3      --data-dir ./line_graph_data \
4      --output-dir ./models
5
6  # Utiliser GAT avec plus d'epochs

```

```

7 python train_gnn.py \
8     -d ./data -o ./models \
9     --model gat \
10    --epochs 300 \
11    --hidden 128
12
13 # Split aleatoire au lieu de spatial
14 python train_gnn.py \
15     -d ./data -o ./models \
16     --no-spatial-split
17
18 # Sans ponderation par observations
19 python train_gnn.py \
20     -d ./data -o ./models \
21     --no-weights
22
23 # Avec Mixed Precision (GPU)
24 python train_gnn.py \
25     -d ./data -o ./models \
26     --amp

```

7.3 Arguments Disponibles

Table 5: Arguments en ligne de commande

Argument	Défaut	Description
-d, -data-dir	./line_graph_data	Répertoire des données
-o, -output-dir	./models	Répertoire de sortie
-model	sage	Type: sage, gat, gc
-hidden	64	Dimension cachée
-layers	3	Nombre de couches
-dropout	0.3	Taux de dropout
-heads	4	Têtes d'attention (GAT)
-epochs	200	Nombre max d'epochs
-lr	0.001	Learning rate
-patience	20	Early stopping patience
-no-spatial-split	False	Split aléatoire
-no-weights	False	Sans pondération
-amp	False	Mixed precision
-loss	mse	Type: mse, mae, huber

8 Code Source : Extraits Clés

8.1 Classe GraphSAGE

```

1 class GraphSAGE(nn.Module):
2     def __init__(self, in_channels, hidden_channels=64,
3                 out_channels=1, num_layers=3, dropout=0.3):
4         super().__init__()
5
6         self.convs = nn.ModuleList()
7         self.bns = nn.ModuleList()
8
9         # Première couche
10        self.convs.append(SAGEConv(in_channels, hidden_channels))

```

```

11     self.bns.append(BatchNorm(hidden_channels))
12
13     # Couches cachees
14     for _ in range(num_layers - 2):
15         self.convs.append(SAGEConv(hidden_channels, hidden_channels))
16         self.bns.append(BatchNorm(hidden_channels))
17
18     # Derniere couche SAGE
19     self.convs.append(SAGEConv(hidden_channels, hidden_channels))
20     self.bns.append(BatchNorm(hidden_channels))
21
22     # MLP de sortie
23     self.mlp = nn.Sequential(
24         nn.Linear(hidden_channels, hidden_channels // 2),
25         nn.ReLU(),
26         nn.Dropout(dropout),
27         nn.Linear(hidden_channels // 2, out_channels)
28     )
29
30     def forward(self, x, edge_index):
31         for i in range(self.num_layers):
32             x = self.convs[i](x, edge_index)
33             x = self.bns[i](x)
34             x = F.relu(x)
35             x = F.dropout(x, p=self.dropout, training=self.training)
36
37         return self.mlp(x).squeeze(-1)

```

Listing 4: Implémentation de GraphSAGE

8.2 Loss Pondérée

```

1 def compute_weighted_loss(predictions, targets, mask, weights, loss_fn):
2     # Appliquer le masque
3     pred_masked = predictions[mask]
4     target_masked = targets[mask]
5
6     # Loss element par element
7     element_loss = loss_fn(pred_masked, target_masked)
8
9     # Ponderer si weights fournis
10    if weights is not None:
11        weights_masked = weights[mask]
12        # Normaliser (log pour atténuer outliers)
13        weights_norm = torch.log1p(weights_masked)
14        weights_norm = weights_norm / weights_norm.sum()
15        return (element_loss * weights_norm).sum()
16
17    return element_loss.mean()

```

Listing 5: Calcul de la loss pondérée

9 Prochaines Étapes

9.1 Phase 4 : Validation et Transfer Learning

1. Validation sur NYC

- Analyser les erreurs par type de route
- Visualiser les prédictions sur carte

- Étudier l'impact du nombre de voisins

2. Transfer vers Marseille

- Appliquer la Phase 1 sur les données OSM de Marseille
- Construire le Line Graph (Phase 2)
- Évaluer le modèle sans ré-entraînement (Zero-Shot)
- Fine-tuning si nécessaire

9.2 Améliorations Possibles

- **Features temporelles** : Ajouter l'heure/jour comme input conditionnel
- **Attention spatiale** : Combiner GAT avec des features de distance
- **Multi-tâche** : Prédire simultanément vitesse et congestion
- **Incertitude** : Ajouter une estimation de l'incertitude (Bayesian GNN)

10 Conclusion

Ce rapport a présenté l'implémentation complète de la Phase 3 du projet de prédiction de trafic urbain. Les contributions principales sont :

1. **Trois architectures GNN** : GraphSAGE (recommandé), GAT, GCN
2. **Split spatial** : Validation de la capacité de généralisation géographique
3. **Gestion des données partielles** : Masquage et pondération
4. **Pipeline complet** : De l'entraînement à la sauvegarde du modèle

Le script `train_gnn.py` est prêt à être exécuté sur les données générées par les Phases 1 et 2.

A Formules Mathématiques Détaillées

A.1 GraphSAGE - Agrégation Mean

$$h_{\mathcal{N}(v)}^{(k)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \quad (9)$$

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, h_{\mathcal{N}(v)}^{(k)} \right) \right) \quad (10)$$

A.2 GAT - Coefficients d'Attention

$$e_{ij} = \text{LeakyReLU} \left(\tilde{a}^T [W h_i \| W h_j] \right) \quad (11)$$

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (12)$$

A.3 GCN - Propagation

$$H^{(l+1)} = \sigma \left(\hat{A} H^{(l)} W^{(l)} \right) \quad (13)$$

avec la matrice d'adjacence normalisée :

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}, \quad \tilde{A} = A + I_N \quad (14)$$

B Nombre de Paramètres

Pour GraphSAGE avec la configuration par défaut :

Couche	Paramètres	Calcul
SAGEConv 1	$(14 + 14) \times 64 + 64$	1856
SAGEConv 2	$(64 + 64) \times 64 + 64$	8256
SAGEConv 3	$(64 + 64) \times 64 + 64$	8256
MLP (Linear 1)	$64 \times 32 + 32$	2080
MLP (Linear 2)	$32 \times 1 + 1$	33
BatchNorm ($\times 3$)	$64 \times 2 \times 3$	384
Total		$\approx 20,865$